

University Course Registration System – A Java Console Application

Introduction

The **University Course Registration System** is a console-based application developed in **Java** that manages students and their course enrolments efficiently. The system allows administrators to add students, create courses, handle enrolments, and monitor course capacities — all through a simple, text-based user interface.

This project aims to provide a **practical demonstration of Object-Oriented Programming (OOP)** principles using Java. It serves as a fundamental prototype for academic management software that can be extended to a real-world university or institutional setting.

The design is modular, interactive, and uses **Java's Collections Framework** for data management. It offers a structured and organized approach to student and course registration processes in educational institutions.

Problem Identification

Challenges in Academic Administration

1. **Manual record-keeping:**

Traditional registration systems rely heavily on paperwork, leading to errors and inefficiency.

2. **Limited tracking and reporting:**

Tracking which students are enrolled in which courses is time-consuming when done manually.

3. **Over-enrolment and data inconsistency:**

Without automated checks, courses may exceed capacity, leading to confusion and mismanagement.

4. **Lack of user-friendly automation tools:**

Small institutions often lack simple software solutions for course and student management.

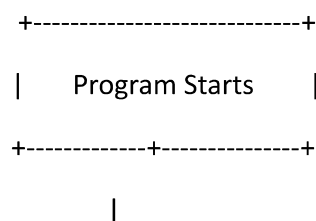
Our Solution: University Course Registration System

This project provides an automated and structured **course registration solution** that:

- Maintains complete details of **students and courses**.
- Prevents **over-enrolment** by enforcing course capacity limits.
- Allows administrators to **add, view, enrol, and drop** students easily.
- Displays reports such as:
 - Students enrolled in each course.
 - Courses a student is currently taking.

By using **Java HashMap and Hash Sets**, the program ensures **fast lookups, data uniqueness**, and **integrity** of operations. It also helps learners understand how OOP principles and collection classes work together to manage relationships between entities.

Flow Diagram:



```

+-----v-----+
|  Show Menu  |
+-----+-----+
|
+-----+-----+
|           |
+--v--+    +---v---+
|Add |      |List |
|Data |      |Data |
+---+---+    +---+---+
|           |
+--v--+    +---v---+
|Enroll/Drop Student |View Info|
+---+---+    +---+---+
|           |
+-----+-----+
|
+-----v-----+
|  Exit  |
+-----+

```

Tools and Technologies Used

Component	Technology/Tool
Programming Language	Java
IDE	IntelliJ IDEA / Eclipse / VS Code
Input Handling	Scanner Class

Component	Technology/Tool
Design Paradigm	Object-Oriented Programming
Operating System	Platform Independent
Type	Console-Based Application

Topics Used in the Project

1. Classes and Objects

- Used to represent entities such as Student, Course, and the Registration System controller.

2. Encapsulation and Abstraction

- Data such as course capacity and student details are encapsulated within respective classes.

3. Java Collections Framework

- HashMap and HashSet used for storing and managing records efficiently.

4. Control Flow Statements

- if, else, and switch used to handle user choices and logical conditions.

5. Loops

- while and for loops used for menu navigation and data iteration.

6. Error Handling

- try-catch blocks used to manage invalid inputs and ensure program stability.

7. Methods (Functions)

- Reusable modular methods (addStudent, addCourse, enrollStudentInCourse) make the program readable and maintainable.

8. Input/Output Operations

- Scanner for user input and System.out.println() for console output.
-

Implementation

Core Functionalities:

1. Add Students:

Adds a new student with a unique student ID and name.

2. Add Courses:

Allows creating new courses with unique IDs, titles, and defined capacity limits.

3. Enroll Student:

Enrolls a student in a specific course after checking for:

- Valid student and course IDs.
- Capacity availability.
- Duplicate enrollment prevention.

4. Drop Student:

Removes a student's enrollment from a specific course.

5. List Students:

Displays all registered students with their IDs and names.

6. List Courses:

Shows all available courses with their capacity and current enrollment counts.

7. View Student Courses:

Lists all courses that a particular student is enrolled in.

8. View Course Students:

Lists all students enrolled in a particular course

Test Case Input		Expected Output
1	Add student (101, Alice)	"Student added successfully."
2	Add course (CSE101, Intro to CS, 2)	"Course added successfully."
3	Enroll student 101 in CSE101	"Enrollment successful."
4	Enroll same student again	"Student already enrolled."
5	Enroll 3rd student in full course	"Course is full."
6	Drop student not enrolled	"Student not enrolled."
7	View all students	Displays list of students.
8	View course students	Shows enrolled student IDs.

Results and Discussion

- The system was tested with multiple students and courses.
- It successfully prevented over-enrollment and duplicate enrollments.
- All menu options performed expected functions, ensuring data integrity.
- The console interface made it easy for users to interact and test different scenarios.

This project proved to be a strong example of **OOP application design** in Java, focusing on **data encapsulation, modularity, and reusability**.

Conclusion

The **University Course Registration System** demonstrates how Java can be used to build a simple yet effective academic management system. It successfully manages students and courses, supports enrollment and removal operations, and presents a clean console-based interface.

The project strengthened understanding of:

- Java Collections Framework
- Object-Oriented Programming concepts
- Exception handling and input validation
- Console-based UI and logical flow control

It provides a strong foundation for future development into a **database-driven or GUI-based application**.

Future Scope

1. **Database Integration:**
Store student and course data persistently using MySQL or SQLite.
2. **Graphical User Interface (GUI):**
Implement JavaFX or Swing for a user-friendly interface.
3. **Login and Authentication System:**
Create separate roles for Admin, Faculty, and Student.
4. **Data Export and Reports:**
Generate PDF or CSV reports for course enrollments and student lists.
5. **Web Application Version:**
Extend the system using Java Servlets, JSP, or Spring Boot.
6. **Email Notifications:**
Notify students about successful enrollments or course updates automatically.